

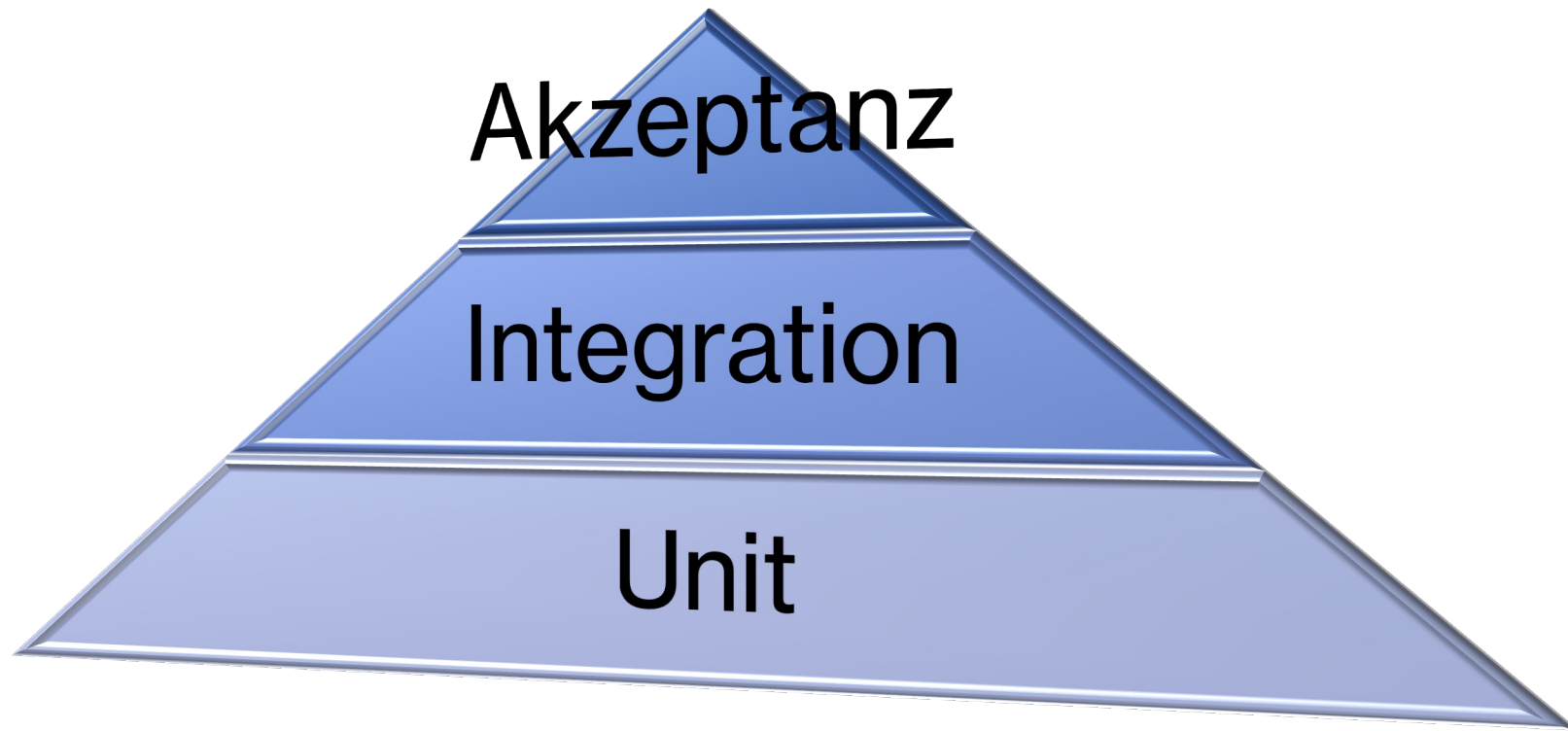
Testing

Unit Tests (Vitest /
Jasmine)

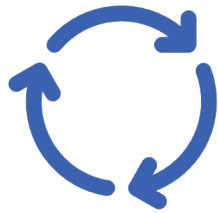
End-to-End (Cypress /
Playwright)



Testpyramide



Angular Test-Runner: Karma/Jasmine (Bestand) – Vitest (Default ab v21)



Karma

TaskRunner (Start Testbed, Reporting Testresults)



Jasmine

API/Framework for the individual Test (Expectations , ...)

Vitest – Default Test-Runner ab Angular 21

// Neue Projekte: ng new nutzt Vitest als Test-Runner

// angular.json

```
"test": {  
  "builder": "@angular/build:unit-test",  
  "options": { "runner": "vitest" }  
}
```

// describe / it / expect / beforeEach bleiben gleich

// Spies: vi.fn() / vi.spyOn() statt jasmine.createSpy()

// Bestandsprojekte (Karma/Jasmine) laufen weiter – siehe folgende Folien

Konfiguration Karma package.json (Bestandsprojekte)

```
"devDependencies": {  
  ..  
  "@types/jasmine": "~4.3.0",  
  "jasmine-core": "~4.6.0",  
  "karma": "~6.4.0",  
  "karma-chrome-launcher": "~3.2.0",  
  "karma-coverage": "~2.2.0",  
  "karma-jasmine": "~5.1.0",  
  "karma-jasmine-html-reporter": "~2.1.0",  
  ..}  
}
```



Konfiguration Karma angular.json (Bestandsprojekte)

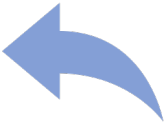
```
"test": {  
  "builder": "@angular-devkit/build-angular:karma",  
  "options": {  
    "polyfills": [  
      "zone.js",  
      "zone.js/testing"  
    ],  
    "tsConfig": "tsconfig.spec.json",  
    "assets": [  
      "src/favicon.ico",  
      "src/assets"  
    ],  
    "styles": [  
      "src/styles.css"  
    ],  
    "scripts": []  
  }  
}
```

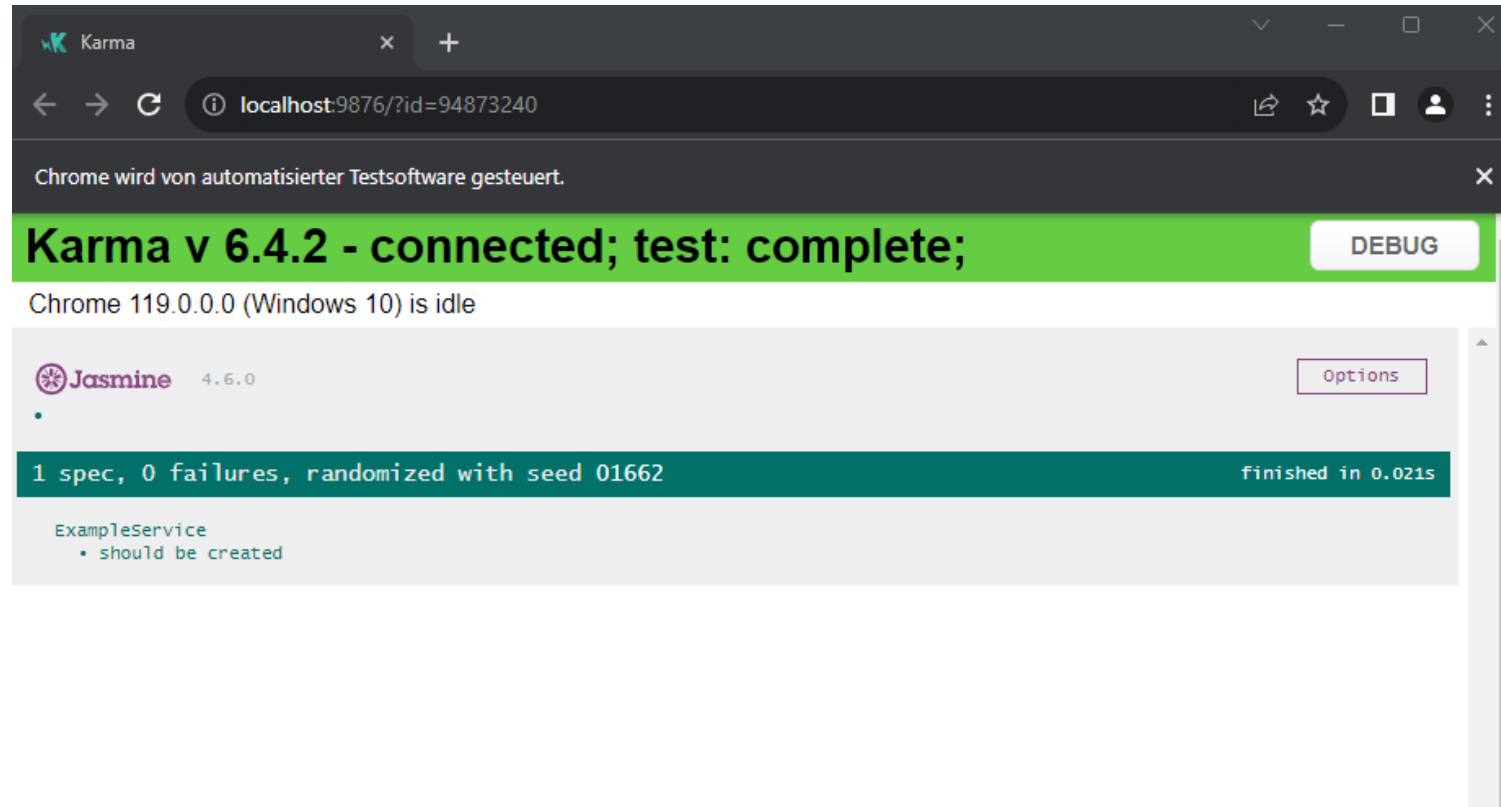


Konfiguration Karma

tsconfig.spec.json

```
/* To learn more about this file see: https://angular.io/config/tsconfig. */
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "outDir": "./out-tsc/spec",
    "types": [
      "jasmine"
    ]
  },
  "include": [
    "src/**/*.spec.ts",
    "src/**/*.d.ts"
  ]
}
```





Test Runner

> ng test

Jasmine Vokabular

Suites – describe(String, function)

- Eine Suite definiert eine Menge von Spezifikationen

Specs – it(string, function)

- 1 Test bestehend aus Titel
- und einer Funktion mit dem Test -> Erwartungen(Expectations).

Expectations expect(actual).toBe(expected)

- Erwartung - technische Implementierung deines Tests → true oder false ausgewertet.

Matchers

- vordefinierte Helferfunktionen für gemeinsame Expectations. z.B.: toBe(expected), toEqual(expected).
- komplette Liste ([GitHub - JamieMason/Jasmine-Matchers: Write Beautiful Specs with Custom Matchers for Jest and Jasmine](#)).

Einfaches Beispiel – Testsubjekt

ExampleService

```
describe('ExampleService', () => {  
  let service: ExampleService;
```

Suite

```
    beforeEach(() => {  
      TestBed.configureTestingModule({});  
      service = TestBed.inject(ExampleService);  
    });
```

Setup

```
    it('should be created', () => {  
      expect(service).toBeTruthy();  
    });  
  });
```

Expectation

Matcher

Setup, Teardown

beforeEach
afterEach

- einmal pro Spec (it)

beforeAll
afterAll

- einmal pro Test-Suite (describe)

Setup initialisiere Testwelt

- beforeEach(() => {
 TestBed.configureTestingModule({}); Setup TestBed mittels Config
 Object

 service = **TestBed**.inject(ExampleService);
});

Expectation

-

Beschreibung

- `it('should be created', () => {
 expect(service).toBeTruthy();
});`

Testcode mit matcher

UnitTest Component

```
@Component({  
  selector: 'app-greeter',  
  templateUrl: './greeter.component.html',  
  styleUrls: ['./greeter.component.css']  
})  
export class GreeterComponent {  
  name = signal("");  
}
```

- `<h1>Hello {{name()}}</h1>`

9/25/26

UnitTests Component - Setup

```
let component: GreeterComponent;
let fixture: ComponentFixture<GreeterComponent>; // Teststeuerobjekt für Component
beforeEach(() => {
  TestBed.configureTestingModule({
    imports: [GreeterComponent] // Standalone Component wird importiert
  });
  fixture = TestBed.createComponent(GreeterComponent);
  component = fixture.componentInstance;
  componentRootElement = fixture.nativeElement; // HTML Element der Component
  debugElement = fixture.debugElement;
  fixture.detectChanges(); // ChangeDetection Trigger
});
```

UnitTests Component – Set

```
it('should show Hello <name>', async () => {  
    component.name.set('Tester'); // Signal setzen  
    fixture.detectChanges(); // Change Detection auslösen  
    await fixture.whenStable(); // Warte bis alle Async Calls beendet sind  
    expect(componentRootElement.querySelector('h1')?.textContent).toBe('Hello Tester');  
    expect(debugElement.query(By.css('h1')).nativeElement.textContent).toBe('Hello Tester');  
});
```

Dependency Injection

```
export class GreeterComponent implements
OnInit {
  name = signal("");
  private readonly service =
inject(GreeterService);
  public ngOnInit(): void {
    this.name.set(this.service.requestName());
  }
}
```

```
@Injectable({ providedIn: 'root' })
export class GreeterService {
  public requestName(): string {
    return 'some example name';
  }
}
```

9/25/26

Mocking Dependencies

```
const greeterServiceSpy = jasmine.createSpyObj('GreeterService', ['requestName']);  
// Vitest: const greeterServiceSpy = { requestName: vi.fn() };  
beforeEach(() => {  
  TestBed.configureTestingModule({  
    imports: [GreeterComponent],  
    providers: [{ provide: GreeterService, useValue: greeterServiceSpy }]  
  });  
  greeterServiceSpy.requestName.and.returnValue('Tester');  
  // Vitest: greeterServiceSpy.requestName.mockReturnValue('Tester');  
  fixture = TestBed.createComponent(GreeterComponent);  
});
```

Async result

```
@for (name of names()); track name) {  
  <p>{{name}}</p>  
}
```

```
export class GreeterComponent {  
  names = signal<string[]>([]);  
  private readonly service = inject(GreeterService);  
  public requestNames(): void {  
    this.service.requestNames().subscribe(result => this.names.set(result));  
  }  
}
```

```
export class GreeterService {  
  
  public requestNames():Observable<string[]>{  
    return //HTTP CALL...  
  }  
}
```

```
});
```

Aync result

```
greeterServiceSpy.requestNames.and.returnValue(of(["Tester1", "Tester2"]));
```

```
...
```

```
it('should show nothing at the beginning', async() => {
```

```
    fixture.detectChanges();
    fixture.whenStable().then(() => {
        expect(componentRootElement.querySelector('p')).toBeNull();
    });
```

```
it('after requesting data 2 p-elements should be visible', async() => {
```

```
    component.requestNames();
    fixture.detectChanges();
    fixture.whenStable().then(() => {
        expect(componentRootElement.querySelectorAll('p')).toHaveLength(2);
    });
```

Throw Error

```
greeterServiceSpy.requestNames.and.returnValue(throwError(() => new Error("TestError")));
```

Trigger Eventhandling

```
@for (name of names(); track name) {  
  <p (click)="setSelectedName(name)">{{name}}</p>  
}
```

```
selected = output<string>();
```

```
public setSelectedName(name: string): void {  
  this.selected.emit(name);  
}
```

```
let selectedName = "";  
component.selected.subscribe(name => (selectedName = name));  
const allPElements = fixture.debugElement.queryAll(By.css('p'));  
allPElements.at(0)?.triggerEventHandler('click');  
expect(selectedName).toEqual('Tester1');
```

Übung 1

- Teste die Klasse CustomerListComponent
 - Mocke den Service und gebe 0, 1, 2 Services zurück
 - Prüfe ob die Liste korrekt aufgebaut
- Teste die currentActiveCustomers Pipe
 - Prüfe mit verschiedenen Werten für „daysInPast“

Cypress

End2End Tests

Installation Cypress

ng e2e

Cannot find "e2e" target for the specified project.

You can add a package that implements these capabilities.

For example:

Cypress: ng add @cypress/schematic

Nightwatch: ng add @nightwatch/schematics

WebdriverIO: ng add @wdio/schematics

Puppeteer: ng add @puppeteer/ng-schematics

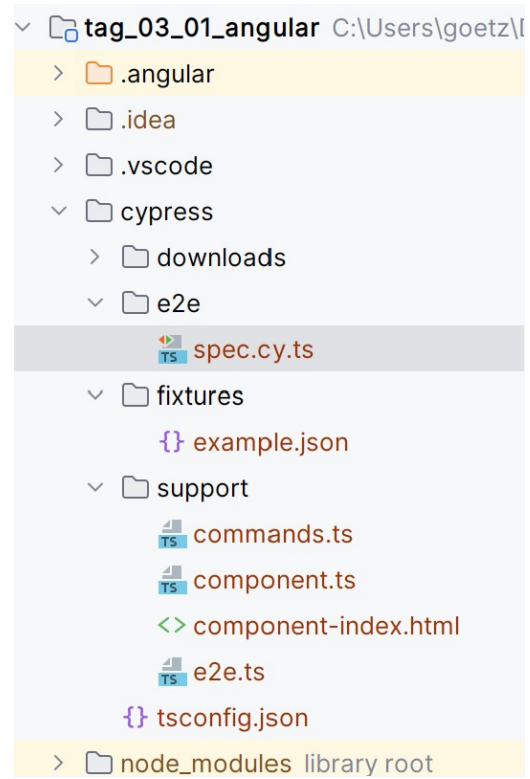
Playwright: ng add playwright-ng-schematics

Would you like to add a package with "e2e" capabilities now? Cypress

i Using package manager: npm

...

Konfig



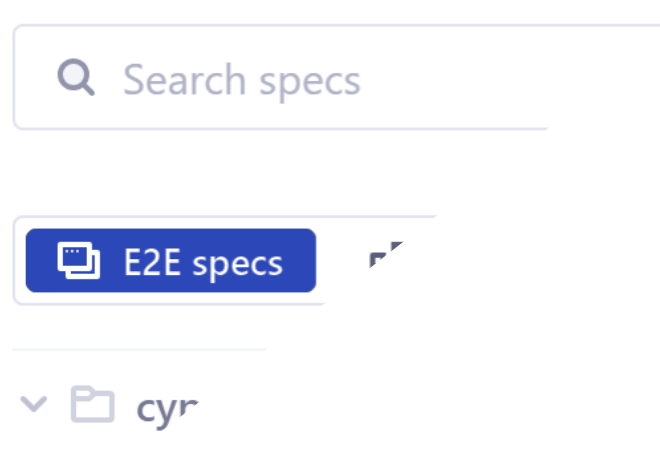
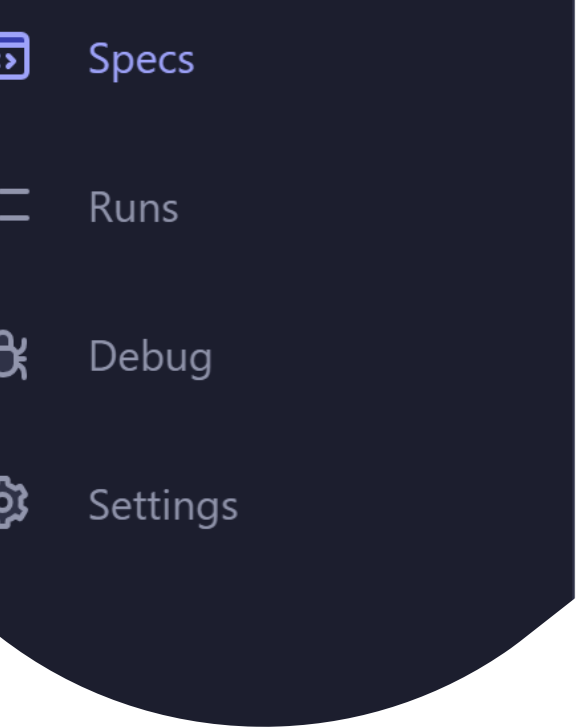
Bestandteile eines Tests

```
describe('My First Test', () => {  
  
  it('Visits the initial project page', () => {  
  
    cy.visit('/')  
    cy.pause();  
    cy.get("li").first().should("have.class", "selected");  
  
  })  
})
```

Suite

Specs

Matcher, Expectation



- Installieren sie Cypress mittels
ng e2e
- Passen sie die generierte Datei
spec.cy.ts
- Ändern sie
 - cy.contains('app is running') in
 - cy.contains('Invoice Manager
App') ab
- Starten sie ng e2e und navigiere
zu
 - spec.cy.ts

Übung 2

Cypress Elements – DSL

- Elemente definieren die structure und Verhalten einer Seite.
- Durch das selektieren und interagieren mit diesen, kann man funktionale Tests schreiben.
- Beispiel ID Selektor
 - `cy.get('#username').type('user@angularseminar.com');`
 - `cy.get('#password').type('password');`
- Beispiel Attribut Selektor
 - `cy.get('[formControlName=username]').type('user@angularseminar.com');`
- Beispiel Element Selektor
 - `cy.get('ul');`

Cypress Elements – Element API

Funktion	Bedeutung
<code>.type()</code>	Ausfüllen eines Textfeldes
<code>.click()</code>	Klicken / Selektieren
<code>.first()</code>	Falls element NodeList, gib erstes
<code>.focus()</code>	Setzt den Fokus auf das Element
<code>.should()</code>	Matcher siehe nächste Folie

Assertions und Matchers

- Die Funktion `should()` an einem Element leitet den Prüfschritt ein
- `cy.get("appCustomerList").should("exist");`
- `cy.get(".selected").should("not.exist");`
- `cy.get(".selected").should("be.visible");`
- `cy.get("li").first().should("have.class","selected");`
- `cy.get("li").first().should("not.have.value",,"Hans");`
- `cy.url().should("contain","greeting");`

Übung 3

- Erstellen Sie eine neue Test Methode `navigation_works`
- Hier soll der erste Eintrag in der `CustomerListe` geklickt werden
- Danach soll geprüft werden, dass
 - Die App auf der Detailseite ist
 - Mittels URL Matcher
 - Und prüfen ob `lastOrder Date` angezeigt wird